Sistema de gestión de gastos personales

Programación 1

Grupo 2

Gobbo Carlos Lautaro 1218112

Caratti Alejandro Maximiliano 1216011

Lozano Marianela Solange 1221723

Luciano Thomas Céspedes 1206567

Sasha Armin Ejarque 1215472

Introducción 3
Objetivo 3
Equipos y Roles 3
Explicación Del Codigo 4
Boceto del menú 6
Conclusión 9
Referencias Bibliográficas 9

Sistema de gestión de gastos

Introducción

El proyecto consiste en el desarrollo de una aplicación por consola en Python que permita al usuario llevar un control y gestión de sus datos personales de manera sencilla. La aplicación permitiría al usuario registrar cada gasto con fecha, descripción, categoría y monto.

Además, brinda opciones para ver gastos cargados, clasificarlos por categorías y obtener un resumen mensual de gastos.

Objetivo

- Desarrollar un sistema MVP que registre y muestre gastos personales desde la consola.
- Incorporar validaciones básicas en la carga de datos (fecha, monto, categoría).
- Implementar un menú simple para el usuario que permita acceder a las principales funcionalidades del programa.
- Organizar el trabajo en equipo dividiendo cargas de trabajo, responsabilidades y fomentar el trabajo grupal.

Equipos y Roles

- **Arquitectura:** *Luciano Thomas Céspedes.*
  Define la organización y estructura de los módulos, organiza tareas y controla las versiones.
- **Desarrollador de funcionalidades:** *Gobbo Carlos Lautaro*
  Implementa la carga de datos del tipo gastos y listados generales.
- **Desarrollador de validaciones:** *Lozano Marianela*
  Implementa la lectura y escritura de archivos CSV (para la generación de resúmenes) y valida las entradas de datos.
- **Desarrollador de reportes:** *Caratti Alejandro Maximiliano*
  Implementa totales por categoría y por rango de fechas.
- **QA y Documentación:** *Sasha Armin Ejarque*
  Redacta la documentación del proyecto y se encarga del testing del programa.

Explicación Del Codigo

El sistema desarrollado implementa un programa para la gestión de gastos personales, estructurado en distintos módulos que se comunican entre sí de manera organizada. Cada uno cumple una función específica dentro del funcionamiento general, lo que permite mantener el código ordenado, legible y fácil de modificar o ampliar en el futuro.

El archivo principal *(main.py)* cumple el rol de punto de inicio. Desde allí se inicia la ejecución del programa, se solicita al usuario que elija o cree un archivo CSV donde se almacenarán los datos, se cargan los registros previos y finalmente se da paso al menú principal. Este menú, definido en *menu.py*, presenta las diferentes opciones del sistema (registrar gastos, listarlos, filtrarlos por categoría o fecha, visualizar totales o generar un resumen mensual) y gestiona la interacción con el usuario de manera dinámica y clara.

El módulo *storage.py* se encarga de todo lo relacionado con la persistencia de datos, es decir, de guardar y recuperar la información en archivos CSV. Allí se encuentran las funciones que crean el archivo en caso de que no exista (*crear_archivo_si_falta()*), que cargan los registros guardados (*cargar_gastos()*) y que los actualizan cuando se realizan cambios (*guardar_gastos()*). Este diseño permite mantener separada la lógica del programa del manejo de archivos, lo que hace el sistema más limpio y robusto.

Por otro lado, *validators.py* reúne todas las funciones de validación, asegurando que los datos que ingresa el usuario sean correctos. Se validan fechas, montos, textos y categorías, evitando errores de formato o valores incoherentes. El programa no avanza hasta que la información sea válida, y muestra mensajes de ayuda cuando hay errores, mejorando la experiencia de uso y reduciendo posibles fallos durante la ejecución.

El módulo *services.py* concentra la parte más lógica del sistema. Allí se realizan las operaciones de procesamiento de datos: agregar gastos nuevos, listar registros, calcular totales o generar resúmenes. Entre las funciones más importantes se encuentran *agregar_gasto(), listar_por_categoria(), totales_por_categoria()* y *totales_por_rango()*. Además, *matriz_mensual()* permite organizar la información por mes y categoría, generando una visión general del comportamiento del gasto a lo largo del año. Este módulo trabaja principalmente con listas y diccionarios, aprovechando la flexibilidad de estas estructuras para manipular información de forma sencilla y eficiente.

El archivo *models.py* contiene las constantes y configuraciones globales del programa, como las categorías disponibles o la ruta base del proyecto. Esto permite mantener centralizados los valores que se utilizan en varios módulos, evitando repeticiones y facilitando la modificación futura.

Desde un punto de vista técnico, el programa utiliza diccionarios para representar cada gasto con sus campos principales (*fecha, desc, cat, monto y moneda*), lo que permite acceder fácilmente a cada dato. El módulo *csv* de Python se encarga de leer y escribir estos datos en el archivo, garantizando compatibilidad y portabilidad. En el proceso de carga, los valores se validan y se transforman antes de incorporarse a la lista principal, evitando inconsistencias.

La validación de datos, basada en la librería *datetime* y en bucles de control, asegura que cada campo sea coherente y que el programa no continúe hasta que el usuario corrija los errores. Esto evita fallos durante los cálculos y mantiene la integridad de los registros.

Por último, el uso de *pathlib.Path* para manejar rutas hace que el programa sea compatible con distintos sistemas operativos sin necesidad de ajustes manuales. Esta decisión técnica simplifica la estructura del proyecto y refuerza la modularidad general del sistema.

Boceto del menú

menu_test.py

```python
def mostrar_menu():
    print("=== Sistema de Gastos ===")
    print("1) Cargar gasto")
    print("2) Listar todos los gastos")
    print("3) Listar gastos por categoría")
    print("4) Ver total por categoría")
    print("5) Ver total por rango de fechas")
    print("0) Salir")


def leer_entero_validado(mensaje, minimo=0, maximo=5):
    while True:
        try:
            x = int(input(mensaje))
            if minimo <= x <= maximo:
                return x
            print(f"Opción fuera de rango ({minimo}-{maximo}).")
        except ValueError:
            print("Debe ingresar un número entero.")


def main():
    opcion = -1
    while opcion != 0:
        mostrar_menu()
        opcion = leer_entero_validado("Opción: ")
        if opcion == 1:
            print("Cargar gasto (fecha/descripcion/categoría/monto)")
        elif opcion == 2:
            print("Listar todos los gastos")
        elif opcion == 3:
            print("Listar por categoría")
        elif opcion == 4:
            print("Total por categoría")
        elif opcion == 5:
            print("Total por rango de fechas")
        else:
            print("Finalizando el programa")


if __name__ == "__main__":
    main()
```

Entrada/Salida por consola

```python
def mostrar_menu():
    print("=== Sistema de Gastos ===")
    print("1) Cargar gasto")
    print("2) Listar todos los gastos")
    print("3) Listar gastos por categoria")
    print("4) Ver total por categoria")
    print("5) Ver total por rango de fechas")
    print("0) Salir")

def leer_entero_validado(mensaje, minimo=0, maximo=5):
    while True:
        try:
            x = int(input(mensaje))
            if minimo <= x <= maximo:
                return x
            else:
                print(f"Opcion fuera de rango ({minimo}-{maximo}).")
        except ValueError:
            print("Debe ingresar un numero entero.")

def main():
    gastos = []  # lista para guardar los gastos
    opcion = -1

    while opcion != 0:
        mostrar_menu()
        opcion = leer_entero_validado("Opcion: ")

        if opcion == 1:
            # Cargar gasto
            fecha = input("Ingrese fecha (dd/mm/aaaa): ")
            descripcion = input("Ingrese descripcion: ")
            categoria = input("Ingrese categoria: ")
            monto = float(input("Ingrese monto: "))
            gastos.append((fecha, descripcion, categoria, monto))
            print(" Gasto registrado con exito.\n")

        elif opcion == 2:
            # Listar todos los gastos
            print("Listado de gastos:")
            for g in gastos:
                print(f"{g[0]} - {g[1]} - {g[2]} - ${g[3]:.2f}")
            print()

        elif opcion == 3:
            # Listar por categoria
            cat = input("Ingrese la categoria a filtrar: ")
            filtrados = [g for g in gastos if g[2].lower() == cat.lower()]
            if filtrados:
                for g in filtrados:
                    print(f"{g[0]} - {g[1]} - {g[2]} - ${g[3]:.2f}")
            else:
                print("No se encontraron gastos en esa categoria.")
            print()

        elif opcion == 4:
            # Total por categoria
            resumen = {}
            for g in gastos:
                resumen[g[2]] = resumen.get(g[2], 0) + g[3]
            for categoria, total in resumen.items():
                print(f"{categoria}: ${total:.2f}")
            print()

        elif opcion == 5:
            # Total por rango de fechas (simplificado)
            print("Funcionalidad en desarrollo...\n")

        else:
            print("Finalizando el programa...")

if __name__ == "__main__":
    main()
```

Salida consola

```
=== Sistema de Gestión de Gastos ===
1) Registrar gasto
2) Listar todos los gastos
3) Listar gastos por categoría
4) Ver total por categoría
5) Ver total por rango de fechas
0) Salir
Seleccione una opción: 1

Fecha (YYYY-MM-DD): 2025-09-19
Descripción: Almuerzo
Categoría: Comida
Monto: 4500
Gasto registrado correctamente.

=== Sistema de Gestión de Gastos ===
1) Registrar gasto
2) Listar todos los gastos
3) Listar gastos por categoría
4) Ver total por categoría
5) Ver total por rango de fechas
0) Salir
Seleccione una opción: 2

Listado de gastos:
---------------------------------------------
1 | 2025-09-19 | Almuerzo   | Comida     | $4500.00
---------------------------------------------
Total de registros: 1

=== Sistema de Gestión de Gastos ===
1) Registrar gasto
2) Listar todos los gastos
3) Listar gastos por categoría
4) Ver total por categoría
5) Ver total por rango de fechas
0) Salir
Seleccione una opción: 4

Ingrese categoría: Comida
Total en Comida: $4500.00

=== Sistema de Gestión de Gastos ===
1) Registrar gasto
2) Listar todos los gastos
3) Listar gastos por categoría
4) Ver total por categoría
5) Ver total por rango de fechas
0) Salir
Seleccione una opción: 0

Programa finalizado.
```

Conclusión

El proyecto logró el objetivo principal de desarrollar un **Sistema de Gestión de Gastos Personales** como un Producto Mínimo Viable (MVP) utilizando Python. Se implementaron con éxito las funcionalidades centrales, incluyendo el registro de gastos (fecha, descripción, categoría y monto), el listado general y el cálculo de totales por categoría.

La estructura del código aplica los conceptos de Programación 1, organizando el flujo de trabajo mediante funciones y validaciones básicas de entrada de usuario. El proyecto también cumplió con el objetivo de fomentar el **trabajo en equipo** y la distribución efectiva de tareas entre los roles definidos.

Referencias Bibliográficas

- **Material de Cátedra:**
  - UADE. (2025). *Apuntes, presentaciones y guías de ejercicios de la materia Programación 1*. Material didáctico interno.
- **Lenguaje de Programación:**
  - Python Software Foundation. (s.f.). *Documentación oficial de Python* (Versión utilizada). Recuperado de [https://docs.python.org/].
- **Herramientas y Estándares:**
  - Python Software Foundation. (s.f.). *Módulo csv – Lectura y escritura de archivos en formato CSV*. Documentación de la librería estándar de Python.
  - Git Community. (s.f.). *Documentación y Tutoriales Oficiales de Git*. Recuperado de [https://git-scm.com/doc].